

iOS 中表格视图和集合视图的实现详解

iOS 项目中表格视图和集合视图的实现详解

一、表格视图（UITableView）的实现

表格视图（UITableView）是 iOS 中用于“列表型数据展示”的核心控件，项目中采用统一的实现规范，涵盖“创建配置、数据源绑定、单元格复用”三大核心环节。

1. 基础创建与配置

项目中通过 **懒加载** 方式创建表格视图，确保控件仅在首次使用时初始化，节省内存资源。创建时需指定 frame、样式，并绑定代理与数据源。

示例代码：

```
- (UITableView *)tableView {
    if (_tableView == nil) {
        // 1. 计算表格视图frame（屏幕宽度，高度 = 屏幕高度 - 导航栏高度 - 标签栏高度）
        CGRect frame = CGRectMake(0, 0,
            self.view.frame.size.width,
            self.view.frame.size.height - IPHONE_TABBARHEIGHT - IPHONE_NAVIGATIONBAR
            _HEIGHT);

        // 2. 创建表格视图（使用普通样式：UITableViewStylePlain）
        _tableView = [[UITableView alloc] initWithFrame:frame
            style:UITableViewStylePlain];

        // 3. 绑定代理与数据源（当前控制器需遵守 UITableViewDelegate、UITableViewDataSource
            协议）
        _tableView.delegate = self;
        _tableView.dataSource = self;

        // （可选）设置其他属性（如分割线、背景色）
        _tableView.separatorStyle = UITableViewCellSeparatorStyleSingleLine;
        _tableView.backgroundColor = [UIColor whiteColor];
    }
    return _tableView;
}
```

```
}  
return _tableView;  
}
```

2. 数据源实现

表格视图的内容由 **数据源 (DataSource)** 提供，项目中实现 `UITableViewDataSource` 协议的核心方法，定义“分区数量、每行数量、单元格内容”。

示例代码：

```
#pragma mark - UITableViewDataSource  
// 1. 返回表格分区数量（此处以数组 self.list 的元素个数作为分区数）  
- (NSInteger)numberOfSectionsInTableView:(UITableView *)tableView {  
    return [self.list count];  
}  
// 2. 返回每个分区的行数（此处以分区对应数组的元素个数作为行数）  
- (NSInteger)tableView:(UITableView *)tableView numberOfRowsInSection:(NSInteger)section  
{  
    return [[self.list objectAtIndex:section] count];  
}  
// 3. 配置每行的单元格（核心方法，负责单元格复用与数据填充）  
- (UITableViewCell *)tableView:(UITableView *)tableView cellForRowAtIndexPath:(  
    NSIndexPath *)indexPath {  
    static NSString *CellIdentifier = @"Cell2"; // 单元格重用标识（静态字符串，避免重复创建）  
  
    // 3.1 从复用队列获取单元格（若无则自动创建）  
    UITableViewCell *cell = [tableView dequeueReusableCellWithIdentifier:CellIdentifier];  
    if (!cell) {  
        cell = [[UITableViewCell alloc] initWithStyle:UITableViewCellStyleSubtitle  
            reuseIdentifier:CellIdentifier];  
    }  
  
    // 3.2 获取当前行对应的数据（假设 self.list 是“分区数组”，每个分区是“行数据数组”）  
    NSArray *sectionData = [self.list objectAtIndex:indexPath.section];  
    NSDictionary *rowData = [sectionData objectAtIndex:indexPath.row];  
  
    // 3.3 填充单元格数据（示例：设置标题、副标题）  
    cell.textLabel.text = rowData[@"title"];  
    cell.detailTextLabel.text = rowData[@"subtitle"];
```

```
return cell;
}
```

3. 单元格复用机制优化

项目通过 **Category（分类）** 封装 `UITableViewCell` 的“Nib 加载”与“重用标识”逻辑，避免硬编码字符串，减少错误并提升复用性。

示例代码（`UITableViewCell+Nib.h`）：

```
#import <UIKit/UIKit.h>
@interface UITableViewCell (Nib)
/**
 * 获取单元格对应的Nib文件（默认Nib文件名与类名一致）
 * @return Nib文件对象
 */
+ (UINib *)nib;
/**
 * 获取单元格的重用标识（命名规则：类名 + "ID"）
 * @return 重用标识字符串
 */
+ (NSString *)cellIdentifier;
@end
```

示例代码（`UITableViewCell+Nib.m`）：

```
#import "UITableViewCell+Nib.h"
@implementation UITableViewCell (Nib)
+ (UINib *)nib {
// 假设Nib文件名与类名相同（如“UserCell.xib”对应“UserCell”类）
return [UINib nibWithNibName:NSStringFromClass(self)
bundle:[NSBundle mainBundle]];
}
+ (NSString *)cellIdentifier {
// 重用标识规则：类名 + "ID"（如“UserCellID”）
return [NSString stringWithFormat:@"%sID", NSStringFromClass(self)];
}
@end
```

使用方式：

```
// 注册自定义单元格（在控制器初始化时调用）
[self.tableView registerNib:[UserCell nib]
forCellReuseIdentifier:[UserCell cellIdentifier]];
// 复用单元格（在 cellForRowAtIndexPath 中）
UserCell *cell = [tableView dequeueReusableCellWithIdentifier:[UserCell cellIdentifier]
forIndexPath:indexPath];
```

二、集合视图（UICollectionView）的实现

集合视图（UICollectionView）支持“网格、瀑布流”等灵活布局，比表格视图更适合复杂视觉排列。项目中实现需额外配置“布局”，核心环节包括“基础配置、数据源代理、动态高度计算”。

1. 基础配置

集合视图的配置比表格视图多一步 **布局设置**（默认使用 UICollectionViewFlowLayout 流式布局），需指定“间距、内边距、滚动方向”等属性，并注册单元格。

示例代码：

```
- (void)setupCollectionView {
// 1. 注册自定义单元格（使用前面定义的 UITableViewCell+Nib 分类，复用逻辑）
[_collectionView registerNib:[HotNewsCLCell nib]
forCellWithReuseIdentifier:[HotNewsCLCell cellIdentifier]];

// 2. 创建并配置流式布局（UICollectionViewFlowLayout）
UICollectionViewFlowLayout *flowLayout = [[UICollectionViewFlowLayout alloc] init];

// 2.1 设置间距（行间距、item 间距）
flowLayout.minimumLineSpacing = 0; // 行间距（垂直滚动时为上下间距）
flowLayout.minimumInteritemSpacing = 0; // item 间距（水平滚动时为左右间距）

// 2.2 设置内边距（集合视图边缘与第一个/最后一个 item 的距离）
flowLayout.sectionInset = UIEdgeInsetsMake(0, 0, 0, 0);

// 2.3 设置滚动方向（垂直滚动：UICollectionViewScrollDirectionVertical）
```

```
flowLayout.scrollDirection = UICollectionViewScrollDirectionVertical;

// 3. 为集合视图应用布局
[self.collectionView setCollectionViewLayout:flowLayout];

// 4. 设置其他属性（背景色、触摸延迟）
self.collectionView.backgroundColor = [UIColor clearColor];
self.collectionView.delaysContentTouches = YES; // 延迟触摸响应，避免误触

// 5. 绑定代理与数据源（当前控制器需遵守 UICollectionViewDelegate、
UICollectionViewDataSource 协议）
self.collectionView.delegate = self;
self.collectionView.dataSource = self;
}
```

2. 数据源和代理实现

集合视图的数据源（UICollectionViewDataSource）与代理（UICollectionViewDelegate）方法，逻辑与表格视图类似，核心是“定义 item 数量、配置 item 内容”。

示例代码：

```
#pragma mark - UICollectionViewDataSource
// 1. 返回分区数量（此处固定为1，即单分区）
- (NSInteger)numberOfSectionsInCollectionView:(UICollectionView *)collectionView {
    return 1;
}

// 2. 返回每个分区的 item 数量（此处以数据数组 self.list 的长度为准）
- (NSInteger)collectionView:(UICollectionView *)collectionView numberOfItemsInSection:(
    NSInteger)section {
    return _list.count;
}

// 3. 配置每个 item 的单元格（核心方法，复用+数据填充）
- (UICollectionViewCell *)collectionView:(UICollectionView *)collectionView
    cellForItemAtIndexPath:(NSIndexPath *)indexPath {
// 3.1 从复用队列获取自定义单元格（HotNewsCLCell 继承自 BaseCLCell）
    HotNewsCLCell *cell = [collectionView dequeueReusableCellWithReuseIdentifier:[
        HotNewsCLCell cellIdentifier]
        forIndexPath:indexPath];
}
```

```

// 3.2 获取当前 item 对应的数据模型（假设 self.list 存储 HotNewModel 实例）
NSInteger row = indexPath.row;
HotNewModel *model = [_list objectAtIndex:row];

// 3.3 填充单元格数据（调用自定义方法 fillCellWithModel:row:delegate:）
[cell fillCellWithModel:model row:row delegate:(id)self];

return cell;
}
#pragma mark - UICollectionViewDelegateFlowLayout
// （可选）设置每个 item 的大小（若需动态高度，需重写此方法）
- (CGSize)collectionView:(UICollectionView *)collectionView layout:(UICollectionViewLayout*)
collectionViewLayout sizeForItemAtIndexPath:(NSIndexPath *)indexPath {
// 此处返回固定大小（动态高度实现见下文）
return CGSizeMake(collectionView.frame.size.width / 2, 200);
}

```

3. 动态高度计算

项目中使用 **第三方库 ARDynamicCacheHeightLayoutCell** 实现集合视图单元格的“动态高度”，即高度根据内容自动调整（如不同长度的文本）。

示例代码：

```

- (CGSize)collectionView:(UICollectionView *)collectionView layout:(UICollectionViewLayout*)
collectionViewLayout sizeForItemAtIndexPath:(NSIndexPath *)indexPath {
// 1. 获取当前 item 对应的数据模型
NSInteger row = indexPath.row;
HotNewModel *model = [_list[row];

// 2. 固定 item 宽度（此处为屏幕宽度，可根据需求调整，如网格布局设为屏幕一半）
CGFloat fixedWidth = [[UIScreen mainScreen] bounds].size.width;

// 3. 使用 ARDynamicCacheHeightLayoutCell 计算动态高度
return [collectionView ar_sizeForCellWithIdentifier:[HotNewsCLCell cellIdentifier]
indexPath:indexPath
fixedWidth:fixedWidth
configuration:^(id cell) {
// 3.1 强转单元格类型
HotNewsCLCell *newsCell = (HotNewsCLCell *)cell;

```

```
// 3.2 填充数据（与 cellForItemAtIndexPath 逻辑一致，确保计算时内容完整）
[newsCell fillCellWithModel:model row:row delegate:(id)self];
}];
}
```

核心原理：通过“预渲染单元格→计算约束高度→缓存结果”的流程，避免重复计算，兼顾动态高度与性能。

三、自定义单元格的实现

项目中为表格 / 集合视图的单元格设计了 **基类 + 子类** 结构，基类封装通用逻辑，子类实现差异化数据填充，提升代码复用性。

1. 集合视图单元格基类（BaseCLCell）

基类 `BaseCLCell` 继承自 `UICollectionViewCell`，封装“数据填充后的约束更新”逻辑，确保单元格布局正确。

示例代码（BaseCLCell.h）：

```
#import <UIKit/UIKit.h>
@interface BaseCLCell : UICollectionViewCell
/**
 * 通用数据填充方法（字典格式数据）
 * @param dict 数据字典
 * @param row 当前行索引
 * @param cv 代理对象（通常为控制器）
 */
- (void)fillCellWithDict:(NSMutableDictionary *)dict row:(NSInteger)row delegate:(id)cv;
@end
```

示例代码（BaseCLCell.m）：

```
#import "BaseCLCell.h"
@implementation BaseCLCell
- (void)awakeFromNib {
    [super awakeFromNib];
}
```

```

// Nib 加载完成后的初始化（如设置子控件默认属性）
}
#pragma mark - 通用数据填充
- (void)fillCellWithDict:(NSMutableDictionary *)dict row:(NSInteger)row delegate:(id)cv {
// 1. 触发约束更新（确保子控件约束生效）
[self setNeedsUpdateConstraints];
[self updateConstraintsIfNeeded];

// 2. 强制布局（确保计算高度时，子控件已完成布局）
[self layoutIfNeeded];
}
@end

```

2. 具体单元格实现（HotNewsCLCell）

子类 `HotNewsCLCell` 继承自 `BaseCLCell`，扩展“模型格式数据填充”方法，实现差异化业务逻辑（如新闻标题、图片展示）。

示例代码（HotNewsCLCell.h）：

```

#import "BaseCLCell.h"
#import "HotNewModel.h"
#import "CommonDelegate.h"
@interface HotNewsCLCell : BaseCLCell
/**
 * 模型格式数据填充（核心方法，适配项目数据模型）
 * @param mod 新闻数据模型（HotNewModel 实例）
 * @param row 当前行索引
 * @param cv 代理对象（遵守 CommonDelegate 协议，处理点击事件）
 */
- (void)fillCellWithModel:(HotNewModel *)mod row:(NSInteger)row delegate:(
UIViewController<CommonDelegate> *)cv;
/**
 * 字典格式数据填充（兼容旧数据格式，可选）
 * @param dict 数据字典
 * @param row 当前行索引
 * @param cv 代理对象
 */
- (void)fillCellWithDict:(NSMutableDictionary *)dict row:(NSInteger)row delegate:(

```



```
UIViewController<CommonDelegate> *)cv;  
@end
```

示例代码（HotNewsCLCell.m，核心逻辑）：

```
#import "HotNewsCLCell.h"  
@interface HotNewsCLCell ()  
// 声明子控件（如新闻标题标签、图片视图）  
@property (nonatomic, weak) IBOutlet UILabel *titleLabel;  
@property (nonatomic, weak) IBOutlet UIImageView *newsImageView;  
@end  
@implementation HotNewsCLCell  
// 模型格式数据填充（核心逻辑）  
- (void)fillCellWithModel:(HotNewModel *)mod row:(NSInteger)row delegate:(  
UIViewController<CommonDelegate> *)cv {  
// 1. 调用基类方法，确保约束更新  
[super fillCellWithDict:nil row:row delegate:cv];  
  
// 2. 填充子控件数据（从模型中取值）  
self.titleLabel.text = mod.title; // 新闻标题  
[self.newsImageView sd_setImageWithURL:[NSURL URLWithString:mod.imageUrl]  
placeholderImage:[UIImage imageNamed:@"placeholder"]]; // 新闻图片（使用  
SDWebImage 加载）  
  
// 3. 绑定代理（如图片点击事件，通过代理回调给控制器）  
self.newsImageView.tag = row;  
UITapGestureRecognizer *tap = [[UITapGestureRecognizer alloc] initWithTarget:self action:@  
selector(imageTapped:)];  
[self.newsImageView addGestureRecognizer:tap];  
self.newsImageView.userInteractionEnabled = YES;  
self.delegate = cv;  
}  
// 字典格式数据填充（兼容旧逻辑，可选）  
- (void)fillCellWithDict:(NSMutableDictionary *)dict row:(NSInteger)row delegate:(  
UIViewController<CommonDelegate> *)cv {  
// 转换字典为模型，复用模型填充逻辑  
HotNewModel *mod = [HotNewModel modelWithDict:dict];  
[self fillCellWithModel:mod row:row delegate:cv];  
}  
// 图片点击事件（通过代理回调）
```

```
- (void)imageTapped:(UITapGestureRecognizer *)tap {  
    if ([self.delegate respondsToSelector:@selector(newsImageTapped:row:)]) {  
        [self.delegate newsImageTapped:self row:tap.view.tag];  
    }  
}  
  
@end
```

四、总结

1. 表格视图 (UITableView) 核心特点

- **创建方式**: 懒加载初始化, 指定 frame 与样式, 绑定代理 / 数据源;
- **数据绑定**: 实现 `UITableViewDataSource` 协议, 定义分区、行数、单元格内容;
- **复用优化**: 通过 Category 封装 Nib 加载与重用标识, 避免硬编码;
- **适用场景**: 单列列表型数据 (如消息列表、设置页面)。

2. 集合视图 (UICollectionView) 核心特点

- **布局配置**: 需额外设置 `UICollectionViewFlowLayout` (间距、滚动方向等);
- **动态高度**: 依赖第三方库 `ARDynamicCacheHeightLayoutCell`, 实现内容自适应高度;
- **适用场景**:

(注: 文档部分内容可能由 AI 生成)